

Sviluppare App Mobile con Flutter: Guida Pratica con Esempio Reale

Questa guida spiega lo sviluppo di applicazioni Flutter attraverso il codice dell'applicazione **Cassa1**, un gestore di bilancio familiare completo e funzionante.

1. Dart: Il Linguaggio di Flutter

Dart è un linguaggio orientato agli oggetti con sintassi familiare a chi già programma.

Concetti fondamentali

```
// Variabili
var nome = 'Mario';           // tipo inferito automaticamente
String cognome = 'Rossi';     // tipo esplicito
final lista = [1, 2, 3];      // immutabile dopo assegnazione
const pi = 3.14;              // costante a compile-time

// Null safety: ogni tipo è non-nullabile di default
String nome = 'test';         // non può essere null
String? maybeNull;             // il ? indica che può essere null
String sicuro = maybeNull ?? 'default'; // ?? fornisce un valore di fallback

// Funzioni
void saluta(String nome) {
  print('Ciao $nome');         // interpolazione con $
}

// Classi e costruttori
class Persona {
  final String nome;
  final int eta;

  Persona({required this.nome, required this.eta}); // Costruttore con named parameters
}

// Async/Await (identico a molti linguaggi moderni)
Future<String> caricaDati() async {
  await Future.delayed(Duration(seconds: 1));
  return 'dati caricati';
}
```

Tipi principali

Tipo	Descrizione	Esempio
List<T>	Lista ordinata	[1, 2, 3]
Map<K,V>	Dizionario chiave-valore	{'a': 1}
Future<T>	Promessa di un valore asincrono	Future<String>
Stream<T>	Flusso di dati asincrono	Stream<int>
DateTime	Data e ora	DateTime.now()

2. Tutto è un Widget

In Flutter, l'interfaccia utente è costruita interamente con codice Dart usando i Widget. Non esistono file di layout separati (come XML o HTML).

Il concetto di Widget

Un Widget è un elemento dell'interfaccia. I Widget possono essere contenitori, elementi grafici, o layout.

```
// Un Widget semplice: un testo centrato
Center(
  child: Text('Hello World'),
)

// Un Widget composto: colonna con piu elementi
Column(
  children: [
    Text('Titolo'),
    SizedBox(height: 16),
    ElevatedButton(
      onPressed: () {},
      child: Text('Premi qui'),
    ),
  ],
)
```

Il metodo `build()`: Il cuore del rendering

Ogni schermata in Flutter implementa un metodo `build()` che ritorna l'albero dei Widget da disegnare:

```
class HomeScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    // Questo metodo viene chiamato ogni volta che il widget deve essere ridisegnato
    return Scaffold(
      appBar: AppBar(title: Text('Home')),
      body: Center(child: Text('Ciao!')),
    );
  }
}
```

`Scaffold` è il widget di base che fornisce la struttura tipica di un'app: `AppBar`, `Body`, `FloatingActionButton`, `Drawer`, ecc.

3. Widget con Stato: `Stateless` vs `Stateful`

I Widget si dividono in due categorie: senza stato (immutabili) e con stato (mutabili).

`StatelessWidget`: Senza stato

Usato per elementi che non cambiano mai dopo la creazione.

```
class Saluto extends StatelessWidget {
```

```

final String nome;

Saluto(this.nome);

@override
Widget build(BuildContext context) {
  return Text('Ciao $nome');
}

```

StatefulWidget: Con stato

Usato quando i dati interni possono cambiare e l'UI deve aggiornarsi.

```

class Contatore extends StatefulWidget {
  @override
  _ContatoreState createState() => _ContatoreState();
}

class _ContatoreState extends State<Contatore> {
  int count = 0;

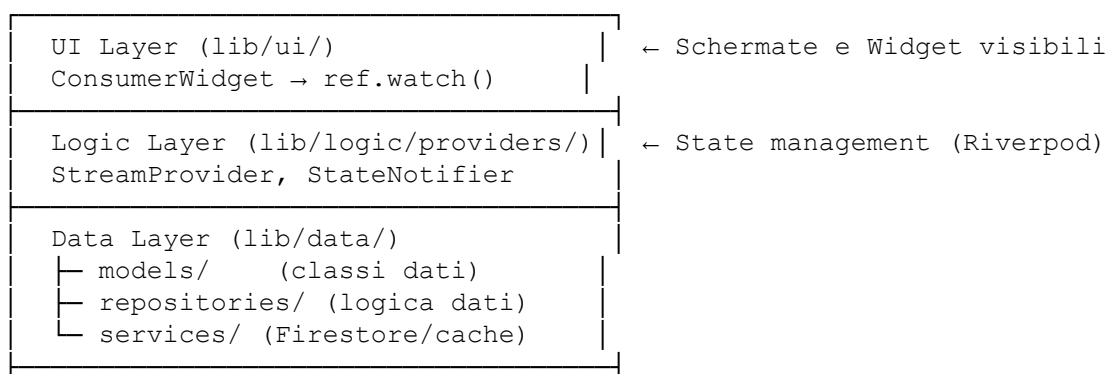
  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text('Count: $count'),
        ElevatedButton(
          onPressed: () {
            setState(() {          // setState() segnala il cambio di stato
              count++;              // e rifa partire il metodo build()
            });
          },
          child: Text('Incrementa'),
        ),
      ],
    );
  }
}

```

Punto chiave: `setState()` dice a Flutter che lo stato è cambiato e che deve ridisegnare il Widget chiamando nuovamente `build()`.

4. Architettura di Cassa1

L'applicazione segue un'architettura a strati (layers) che separa le responsabilità:



Flusso dei dati (dal database all'interfaccia)

```
Firestore → FirebaseService (getTransactionsStream())  
    ↓  
TransactionRepository (dual-source: cache + Firestore)  
    ↓  
transactionsProvider (StreamProvider)  
    ↓  
UI: ref.watch(transactionsProvider) → visualizza la lista
```

5. I Modelli (Models): La Struttura dei Dati

I modelli definiscono la forma dei dati dell'applicazione. Sono classi Dart con supporto per la serializzazione JSON.

Esempio: `AppTransaction` (`lib/data/models/transaction.dart`)

```
enum TransactionType { income, expense, transfer, anticipi }  
  
class AppTransaction {  
  final String id;  
  final TransactionType type;  
  final double amount;  
  final DateTime date;  
  final String? note;  
  final String? subjectId;  
  final String? fromSubjectId;  
  final String? toSubjectId;  
  final DateTime createdAt;  
  
  AppTransaction({  
    required this.id,  
    required this.type,  
    required this.amount,  
    required this.date,  
    this.note,  
    this.subjectId,  
    this.fromSubjectId,  
    this.toSubjectId,  
    required this.createdAt,  
  });  
  
  // Da JSON a oggetto  
  factory AppTransaction.fromJson(Map<String, dynamic> json) {  
    return AppTransaction(  
      id: json['id'] as String,  
      type: TransactionType.values.firstWhere(  
        (e) => e.name == json['type'],  
      ),  
      amount: (json['amount'] as num).toDouble(),  
      date: DateTime.parse(json['date'] as String),  
      note: json['note'] as String?,  
      subjectId: json['subjectId'] as String?,  
      entryId: json['entryId'] as String?,  
      fromSubjectId: json['fromSubjectId'] as String?,  
      toSubjectId: json['toSubjectId'] as String?,  
    );  
  }  
}
```

```

        createdAt: DateTime.parse(json['createdAt'] as String),
    );
}

// Da oggetto a JSON
Map<String, dynamic> toJson() {
    return {
        'id': id,
        'type': type.name,
        'amount': amount,
        'date': date.toIso8601String(),
        'note': note,
        'subjectId': subjectId,
        'entryId': entryId,
        'fromSubjectId': fromSubjectId,
        'toSubjectId': toSubjectId,
        'createdAt': createdAt.toIso8601String(),
    };
}
}

```

Nota: `factory` è un costruttore che può restituire un'istanza esistente o crearne una nuova (simile a un metodo di classe che ritorna l'oggetto). `Map<String, dynamic>` è l'equivalente di un dizionario generico.

Altri modelli in Cassa1

- **Subject:** rappresenta una persona (nome, icona)
- **Group:** raggruppa le voci di spesa/entrata per tipo (`income` o `expense`)
- **Entry:** una voce specifica (es. "Spesa", "Stipendio") collegata a un Group

6. Services: Accesso al Database

I Services contengono la logica di accesso ai dati esterni. In Cassa1, `FirebaseService` comunica con Firestore.

FirebaseService (lib/data/services/firebase_service.dart)

```

class FirebaseService {
    final _db = FirebaseFirestore.instance;

    // Stream di transazioni in tempo reale
    Stream<List<AppTransaction>> getTransactionsStream() {
        return _db
            .collection('transactions')
            .orderBy('date', descending: true)
            .snapshots()
            .map((snapshot) => snapshot.docs
                .map((doc) => AppTransaction.fromJson(doc.data()))
                .toList());
    }

    // Operazioni CRUD
    Future<void> addTransaction(AppTransaction t) =>
        _db.collection('transactions').doc(t.id).set(t.toJson());

    Future<void> updateTransaction(AppTransaction t) =>
        _db.collection('transactions').doc(t.id).update(t.toJson());

    Future<void> deleteTransaction(String id) =>
        _db.collection('transactions').doc(id).delete();
}

```

```
}
```

Concetti chiave:

- `Stream` in Dart = un flusso di dati asincrono a cui ci si può "iscrivere"
 - `.snapshots()` ascolta i cambiamenti in tempo reale su Firestore
 - `.map()` trasforma i dati (come nelle funzioni di ordine superiore)
 - `set()` = inserisci o sovrascrivi un documento
 - `update()` = aggiorna campi specifici di un documento
-

7. Repositories: Cache + Network

Il pattern Repository nasconde la complessità di "da dove arrivano i dati" (cache locale vs rete).

TransactionRepository (lib/data/repositories/transaction_repository.dart)

```
class TransactionRepository {
  final FirebaseService firebaseService;
  final CacheService cacheService;

  TransactionRepository(this.firebaseService, this.cacheService);

  Stream<List<AppTransaction>> getTransactions() async* {
    // 1. Prima mostra la cache (esperienza utente veloce)
    final cached = cacheService.getTransactions();
    if (cached != null) {
      final decoded = jsonDecode(cached) as List;
      yield decoded.map((j) => AppTransaction.fromJson(j)).toList();
    }
    // 2. Poi aggiorna con Firestore
    await for (final transactions in firebaseService.getTransactionsStream()) {
      final json = jsonEncode(transactions.map((t) => t.toJson()).toList());
      await cacheService.saveTransactions(json);
      yield transactions;
    }
  }
}
```

Pattern: `async* + yield` = generatore asincrono che produce una sequenza di valori nel tempo.

8. State Management: Riverpod

Riverpod è una libreria per gestire lo stato globale dell'applicazione in modo reattivo. Permette di rendere i dati disponibili in tutto il tree dei Widget e di notificare i Widget quando i dati cambiano.

I Provider: Dichiarare le dipendenze

```
// Provider di base: istanza singleton di FirebaseService
final firebaseServiceProvider = Provider<FirebaseService>((ref) => FirebaseService());

// Provider di repository: dipende da firebaseServiceProvider
final transactionRepositoryProvider = Provider<TransactionRepository>((ref) {
  return TransactionRepository(
```

```

        ref.watch(firebaseServiceProvider),
        CacheService(ref.watch(sharedPreferencesProvider)),
    );
});

// StreamProvider: espone uno stream di dati reattivi
final transactionsProvider = StreamProvider<List<AppTransaction>>((ref) {
    return ref.watch(transactionRepositoryProvider).getTransactions();
});

```

ConsumerWidget: Ascoltare i Provider

```

class TransactionList extends ConsumerWidget {
    @override
    Widget build(BuildContext context, WidgetRef ref) {
        final txsAsync = ref.watch(transactionsProvider);

        return txsAsync.when(
            loading: () => CircularProgressIndicator(), // Mentre carica
            error: (e, _) => Text('Errore: $e'), // Se errore
            data: (transactions) => ListView.builder( // I dati!
                itemCount: transactions.length,
                itemBuilder: (context, index) => ListTile(
                    title: Text('€ ${transactions[index].amount}'),
                ),
            ),
        );
    }
}

```

Differenza tra `ref.watch()` e `ref.read()`:

- `watch` = ascolta i cambiamenti (usato nel metodo `build()`)
- `read` = leggi il valore una volta sola (usato in callback come `onPressed`)

Gerarchia dei Provider in Cassa1

```

firebaseServiceProvider (istanza di FirebaseService)
    ↓
sharedPreferencesProvider (istanza di SharedPreferences)
    ↓
transactionRepositoryProvider (TransactionRepository)
    ↓
transactionsProvider (StreamProvider<List<AppTransaction>>)
    ↓
UI: ref.watch(transactionsProvider) → aggiorna l'interfaccia

```

9. L'UI: Schermate e Widget di Cassa1

Struttura tipica di una Schermata

```

class AllTransactionsScreen extends ConsumerWidget {
    @override
    Widget build(BuildContext context, WidgetRef ref) {
        final txs = ref.watch(transactionsProvider);
        final subjects = ref.watch(subjectsProvider);
        final groups = ref.watch(groupsProvider);
    }
}

```

```

final entries = ref.watch(entriesProvider);

return Scaffold(
  appBar: AppBar(title: Text('Tutti i movimenti')),
  body: txs.when(
    loading: () => Center(child: CircularProgressIndicator()),
    error: (e, _) => Center(child: Text('Errore: $e')),
    data: (transactions) => ListView.builder(
      itemCount: transactions.length,
      itemBuilder: (context, index) => _buildTile(transactions[index]),
    ),
  ),
);
}
}

```

Widget principali usati in Cassa1

Widget	Scopo
Scaffold	Struttura pagina base (AppBar, Body, Drawer)
AppBar	Barra superiore con titolo e azioni
ListView / ListView.builder	Lista scrollabile di elementi
Column / Row	Layout verticale / orizzontale
Container	Box con padding, margini, colore
Card	Carta con ombra e bordi arrotondati
ListTile	Riga strutturata (icona, titolo, sottotitolo, trailing)
DropDownButton	Menù a tendina per selezione
AlertDialog	Popup modale per conferme o form
TextField	Campo di input testuale
InkWell	Area cliccabile con effetto ripple
SingleChildScrollView	Scroll per contenuti che eccedono lo spazio

10. Form e Validazione: Aggiungere una Transazione

Ecco come funziona il form di aggiunta in `all_transactions_screen.dart`:

```

void _showAddDialog(BuildContext context, WidgetRef ref) {
  final amountController = TextEditingController();
  final noteController = TextEditingController();
  TransactionType selectedType = TransactionType.expense;
  DateTime selectedDate = DateTime.now();
  String? selectedSubjectId;

  showDialog(
    context: context,
    builder: (dialogContext) => StatefulBuilder(
      builder: (dialogContext, setState) => AlertDialog(
        title: Text('Nuovo movimento'),
        content: SingleChildScrollView(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
              DropdownButton<TransactionType>(
                value: selectedType,
                items: [
                  DropdownMenuItem(value: TransactionType.income, child: Text('Entrata')),

```

```

        DropdownMenuItem(value: TransactionType.expense, child: Text('Uscita')),
        // ...
    ],
    onChanged: (value) => setState(() => selectedType = value!),
),
TextField(
    controller: amountController,
    keyboardType: TextInputType.number,
    decoration: InputDecoration(labelText: 'Importo *'),
),
// ... altri campi
],
),
),
actions: [
    TextButton(
        onPressed: () {
            final amount = double.tryParse(amountController.text);
            if (amount == null || amount <= 0) {
                ScaffoldMessenger.of(context).showSnackBar(
                    SnackBar(content: Text('Importo non valido'), backgroundColor: Colors.red),
                );
                return;
            }
            final repo = ref.read(transactionRepositoryProvider);
            repo.add(AppTransaction(
                id: DateTime.now().millisecondsSinceEpoch.toString(),
                type: selectedType,
                amount: amount,
                date: selectedDate,
                subjectId: selectedSubjectId,
                entryId: selectedEntryId,
                createdAt: DateTime.now(),
            ));
            Navigator.pop(dialogContext);
        },
        child: Text('Salva'),
    ),
],
),
),
);
}

```

Nota su TextEditingController: gestisce il testo di un `TextField` (leggere/impostare il valore, ascoltare i cambiamenti). Simile al concetto di "binding" in altri framework.

11. Calcolo del Bilancio

Il calcolo avviene direttamente nel Widget, filtrando e sommando le transazioni:

```

// Dentro il metodo build()
final income = transactions
    .where((t) => t.type == TransactionType.income)
    .fold(0.0, (sum, t) => sum + t.amount);

final expense = transactions
    .where((t) => t.type == TransactionType.expense)
    .fold(0.0, (sum, t) => sum + t.amount);

final transferIn = transactions

```

```

        .where((t) => t.type == TransactionType.transfer && t.toSubjectId != null)
        .fold(0.0, (sum, t) => sum + t.amount);

final transferOut = transactions
    .where((t) => t.type == TransactionType.transfer && t.fromSubjectId != null)
    .fold(0.0, (sum, t) => sum + t.amount);

final anticipi = transactions
    .where((t) => t.type == TransactionType.anticipi)
    .fold(0.0, (sum, t) => sum + t.amount);

final balance = income - expense + transferIn - transferOut;

```

Le anticipi (anticipi) sono escluse dal saldo e mostrate separatamente.

Metodi funzionali usati:

- `where()` = filtra gli elementi secondo una condizione
- `fold()` = riduce una lista a un singolo valore accumulando

12. Esportazione Dati: Un Esempio di Service

Il servizio di esportazione mostra come combinare piu tecnologie:

```

// lib/data/services/export_service.dart
class ExportService {
  static Future<String> generateCsv({
    required List<AppTransaction> transactions,
    required List<Subject> subjects,
    required List<Entry> entries,
    required List<Group> groups,
  }) async {
    final dateFormat = DateFormat('dd/MM/yyyy');
    final rows = <List<dynamic>>[
      ['Data', 'Tipo', 'Soggetto', 'Voce', 'Importo', 'Nota'], // Header
    ];

    for (final t in transactions) {
      final subject = subjects.where((s) => s.id == t.subjectId).firstOrNull;
      final entry = entries.where((e) => e.id == t.entryId).firstOrNull;
      rows.add([
        dateFormat.format(t.date),
        _transactionTypeLabel(t.type),
        subject?.name ?? '',
        entry?.name ?? '',
        t.amount.toStringAsFixed(2),
        t.note ?? '',
      ]);
    }

    return const ListToCsvConverter().convert(rows); // Usa package 'csv'
  }

  static Future<void> exportAndShare({...}) async {
    final csv = await generateCsv(...);
    final dir = await getTemporaryDirectory(); // path_provider
    final file = File('${dir.path}/export.csv');
    await file.writeAsString(csv);

    await Share.shareXFiles([XFile(file.path)]); // share_plus
  }
}

```

```
}  
}
```

Packages usati:

- csv = generazione stringhe CSV
 - path_provider = accesso alle directory del dispositivo
 - share_plus = apertura del menu di condivisione di sistema
-

13. Navigation: Router e GoRouter

La navigazione tra schermate è gestita da `go_router`, che definisce i percorsi (routes) dell'applicazione.

Configurazione del Router (`lib/ui/router/app_router.dart`)

```
final appRouterProvider = Provider<GoRouter>((ref) {  
  return GoRouter(  
    initialLocation: '/',  
    redirect: (context, state) {  
      // Se non autenticato → /login  
      final user = ref.watch(authStateProvider).value;  
      if (user == null) return '/login';  
      return null;  
    },  
    routes: [  
      GoRoute(path: '/', builder: (_, __) => HomeScreen()),  
      GoRoute(path: '/login', builder: (_, __) => LoginScreen()),  
      GoRoute(  
        path: '/subjects/:id',  
        builder: (context, state) {  
          final id = state.params['id']!; // Parametro della route  
          return SubjectDetailScreen(subjectId: id);  
        },  
      ),  
    ],  
  );  
});
```

Navigare tra le schermate

```
// Sostituisce la schermata corrente (come window.location = ...)  
context.go('/subjects/123');  
  
// Aggiunge alla pila di navigazione (puoi tornare indietro)  
context.push('/subjects/123');
```

14. Inizializzazione dell'App (`main.dart`)

Il punto di ingresso dell'applicazione configura tutti i servizi necessari prima di avviare l'UI.

```
void main() async {  
  WidgetsFlutterBinding.ensureInitialized(); // Inizializza i binding di Flutter
```

```

await Firebase.initializeApp(                                // Inizializza Firebase
  options: DefaultFirebaseOptions.currentPlatform,
);

await initializeDateFormatting('it_IT', null); // Locale italiano per date

final prefs = await SharedPreferences.getInstance();

runApp(
  ProviderScope(                                             // Entry point di Riverpod
    overrides: [
      sharedPreferencesProvider.overrideWithValue(prefs),
    ],
    child: MyApp(),
  ),
);
}

class MyApp extends ConsumerWidget {
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final router = ref.watch(appRouterProvider);
    final themeMode = ref.watch(themeModeProvider);

    return MaterialApp.router(
      title: 'Cassa Famiglia',
      theme: buildLightTheme(),
      darkTheme: buildDarkTheme(),
      themeMode: themeMode,
      routerConfig: router,
    );
  }
}

```

15. Temi e Stili

Cassa1 usa Material Design 3 con colori personalizzati e il font Poppins.

Definizione dei Colori (lib/utils/constants.dart)

```

class AppColors {
  static const primary = Color(0xFF6C63FF);
  static const incomeColor = Color(0xFF4CAF50);
  static const expenseColor = Color(0xFFFF44336);
  static const transferColor = Color(0xFF2196F3);
  static const anticipiColor = Color(0xFFFFF9800);
}

```

Uso dei Colori nel Codice

```

// Invece di colori hardcoded, usa il ColorScheme del tema
Color amountColor = Theme.of(context).colorScheme.primary;

// Oppure definisci colori semantici
if (t.type == TransactionType.income) {
  amountColor = AppColors.incomeColor;
}

```

Nota: `Theme.of(context)` accede al tema corrente (light o dark) e fornisce colori adattivi automaticamente.

16. Checklist per Iniziare con Flutter

1. **Widget = elementi UI:** costruisci l'interfaccia componendo Widget gerarchicamente
 2. **build() e il template:** ogni Widget ha un metodo `build()` che ritorna l'albero UI
 3. **Stato:** `setState()` per Widget con stato locale, Riverpod per stato globale reattivo
 4. **async*/yield:** per creare stream di dati (come i generatori asincroni)
 5. **Null safety:** tutti i tipi sono non-nullable di default; usa `?` per nullable
 6. **Packages:** `pubspec.yaml` e `flutter pub get` per gestire le dipendenze
 7. **Hot reload:** premi `r` nel terminale per ricaricare l'app senza perdere lo stato
 8. **StreamBuilder/ConsumerWidget:** ascoltano flussi di dati e aggiornano l'UI automaticamente
-

17. Esercizio: Aggiungere una Nuova Feature

Immagina di voler aggiungere un campo "Categoria" alle transazioni. Ecco i passi:

1. **Modello:** Aggiungi `final String? categoryId; in AppTransaction`
2. **JSON:** Aggiorna `fromJson()` e `toJson()`
3. **Firestore:** Il campo apparirà automaticamente nei documenti
4. **Form:** Aggiungi un `DropDownButton` in `_showAddDialog()`
5. **UI:** Mostra la categoria nel `ListTile`

Il flusso è: modello → provider → form → widget.

Conclusione

Cassa1 è un esempio compatto di app Flutter reale con:

- **4 modelli dati** (Transaction, Subject, Entry, Group)
- **Firebase** come backend (Auth + Firestore in tempo reale)
- **Riverpod** per state management reattivo
- **UI responsiva** con Material Design 3
- **Feature reali:** calcolo bilancio, chiusura mensile, esportazione CSV
- **Input vocale con AI:** creazione transazioni tramite comando vocale e parsing intelligente

L'intero codice è in `lib/` — esplora i file seguendo l'architettura a strati descritta sopra per comprendere ogni componente.

18. Input Vocale con AI: Speech-to-Text e OpenRouter

Cassa1 permette di creare transazioni parlando al microfono. Il flusso unisce riconoscimento vocale locale e intelligenza artificiale cloud.

Flusso completo

Microfono → speech_to_text (locale) → testo trascritto
↓
OpenRouter API (Gemini Flash) → parsing JSON strutturato
↓
Dialogo di conferma → form pre-compilato → salvataggio transazione

Pacchetti utilizzati

Pacchetto	Scopo
speech_to_text	Riconoscimento vocale tramite i motori di sistema (Google/Apple)
http	Chiamata REST a OpenRouter API

Servizio VoiceTransactionService (lib/data/services/voice_transaction_service.dart)

Il servizio coordina la trascrizione e l'analisi AI:

```
class VoiceTransactionService {
  final SpeechToText _speech = SpeechToText();
  final String openRouterApiKey;

  Future<bool> initialize() async {
    return await _speech.initialize(
      onError: (error) => debugPrint('Speech error: $error'),
    );
  }

  Future<void> startListening({
    required Function(String) onResult,
    required Function() onComplete,
  }) async {
    await _speech.listen(
      onResult: (result) {
        onResult(result.recognizedWords);
        if (result.finalResult) onComplete();
      },
      localeId: 'it_IT', // italiano
      listenMode: ListenMode.confirmation,
    );
  }
}
```

Parsing con AI (OpenRouter)

Una volta ottenuto il testo trascritto, viene inviato a OpenRouter con un prompt che include:

1. **La lista dei soggetti disponibili** (con ID e nome)
2. **La lista delle voci** raggruppate per gruppo
3. **Il testo trascritto** dell'utente

L'AI (modello google/gemini-2.0-flash-001) restituisce un JSON strutturato:

```
{
  "type": "expense",
  "amount": 15.0,
  "subjectName": "Massimo",
  "entryName": "Pranzo",
  "note": "",
}
```

```
"confidence": 0.9
}
```

Il servizio mappa poi i nomi agli ID reali tramite fuzzy matching:

```
Subject? _fuzzyFindSubject(String name, List<Subject> subjects) {
  final lowerName = name.toLowerCase();
  // Exact match
  var match = subjects.where((s) => s.name.toLowerCase() == lowerName);
  if (match.isNotEmpty) return match.first;
  // Contains match
  match = subjects.where((s) => s.name.toLowerCase().contains(lowerName));
  if (match.isNotEmpty) return match.first;
  return null;
}
```

Widget VoiceTransactionDialog (lib/ui/widgets/voice_transaction_dialog.dart)

Il dialogo gestisce l'intero ciclo vocale:

```
class VoiceTransactionDialog extends ConsumerStatefulWidget {
  // Stati del dialogo:
  // - idle: pronto, mostra icona microfono
  // - listening: sta registrando (icona rossa, mostra testo in tempo reale)
  // - processing: invia a OpenRouter, mostra spinner
  // - confirm: mostra i dati parsati, l'utente conferma
  // - error: errore, permette retry
}
```

Integrazione nell'UI

I pulsanti microfono sono stati aggiunti nelle AppBar di:

- `subject_detail_screen.dart` (schermata dettaglio soggetto)
- `all_transactions_screen.dart` (tutti i movimenti)

```
IconButton(
  icon: const Icon(Icons.mic),
  tooltip: 'Nuova transazione vocale',
  onPressed: () => _showVoiceDialog(context, ref, subject),
),
```

Il metodo `_showVoiceDialog` apre il dialogo vocale, attende il risultato e poi pre-compila il form di aggiunta:

```
void _showVoiceDialog(BuildContext context, WidgetRef ref, Subject subject) async {
  final result = await showDialog<VoiceTransactionResult>(
    context: context,
    builder: (dialogContext) => VoiceTransactionDialog(
      subjects: widget.subjects,
      entries: widget.entries,
      groups: widget.groups,
    ),
  );

  if (result != null && !result.isError) {
    _showAddDialogFromVoice(context, ref, result);
  }
}
```

```
}
```

Configurazione necessaria

1. **Permessi Android** (android/app/src/main/AndroidManifest.xml):

```
<uses-permission android:name="android.permission.RECORD_AUDIO"/>
```

2. **API Key:** impostare la variabile d'ambiente prima di avviare l'app:

```
flutter run -d edge --dart-define=OPENROUTER_API_KEY=tua_chiave_api
```

Esempio di utilizzo

L'utente tocca il microfono, dice: *"uscita Massimo pranzo 15 euro"*, l'app:

1. Trascrive in tempo reale col microfono
2. Invia il testo a OpenRouter: "Riconosci tipo=expense, soggetto=Massimo, voce=Pranzo, amount=15"
3. Mostra la conferma: Tipo=Uscita, Importo=€15.00, Soggetto=Massimo, Voce=Pranzo (confidenza 90%)
4. L'utente conferma → la transazione viene salvata

Vantaggio del parsing AI: gestisce automaticamente sinonimi ("pranzo" → voce nel gruppo "Spese alimentari"), numeri scritti ("quindici" → 15.0), e ordine delle parole libero.